Sistemas Operacionais

Sincronização de processos e threads
Fundamentos, semáforos, monitores e deadlocks

Prof. Frederico Luís Cabral

Universidade Federal de Ciência e Inovação do Rio de Janeiro (UFCIRJ)

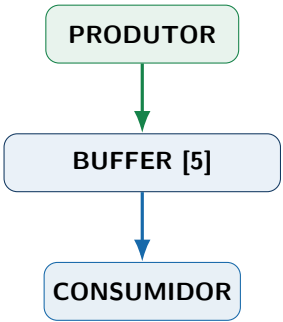
Bacharelado em Ciência da Computação

github.com/fredluiscabral/UFCIRJ/tree/main/SO

Outubro e novembro de 2026

Uma situação real: produzir e consumir dados

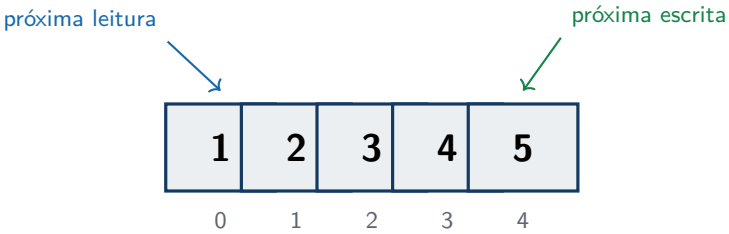
- Uma **thread produtora** gera itens.
- Uma **thread consumidora** processa itens.
- Ambas acessam um **buffer compartilhado**.
- O buffer tem **capacidade finita**: 5 posições.



Pergunta para a turma

O que pode acontecer quando as duas threads executam concorrentemente?

O buffer circular



- Índice de escrita: `entrada = (entrada + 1) % 5`
- Índice de leitura: `saida = (saida + 1) % 5`
- Ao chegar ao fim, cada índice **volta à posição zero**.

Questão

O índice circular, sozinho, impede a sobrescrita de dados não consumidos?

Primeira implementação: dados compartilhados

```c
#define TAM_BUFFER 5
#define TOTAL_ITENS 20

int buffer[TAM_BUFFER] = {0};
int entrada = 0;
int saida = 0;
```

- `buffer`: vetor visível às duas threads.
- `entrada`: posição onde o produtor escreverá.
- `saida`: posição de onde o consumidor lerá.
- Valor `0`: marcador didático de posição vazia (itens válidos: 1 a 20).

O código completo está no arquivo `produtor_consumidor.c` do projeto.

O produtor: escreve sem verificar espaço livre

```c
void *produtor(void *arg) {
    for (int item = 1; item <= TOTAL_ITENS; ++item) {
        usleep(100000);           // 100 ms
        int pos = entrada;
        buffer[pos] = item;       // escreve SEM verificar!
        entrada = (entrada + 1) % TAM_BUFFER;
        printf("PRODUZ %d -> posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O produtor pode escrever em uma posição que ainda guarda um item não consumido.

O consumidor: lê sem verificar se há item

```
void *consumidor(void *arg) {
    for (int i = 0; i < TOTAL_ITENS; ++i) {
        usleep(150000);            // 150 ms
        int pos = saida;
        int item = buffer[pos];    // le SEM verificar!
        buffer[pos] = 0;           // limpa visualmente
        saida = (saida + 1) % TAM_BUFFER;
        printf("CONSOME %d <- posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O consumidor pode ler uma posição vazia, ou receber um item diferente daquele esperado.

As duas threads executam concorrentemente

```
int main(void) {
    pthread_t tprod, tcons;

    pthread_create(&tprod, NULL, produtor, NULL);
    pthread_create(&tcons, NULL, consumidor, NULL);

    pthread_join(tprod, NULL);
    pthread_join(tcons, NULL);
    return 0;
}
```

- `pthread_create`: inicia cada thread.
- `pthread_join`: faz a thread principal aguardar o término.
- **Nenhuma dessas chamadas protege o buffer.**

Evolução possível do buffer (intercalação ilustrativa)

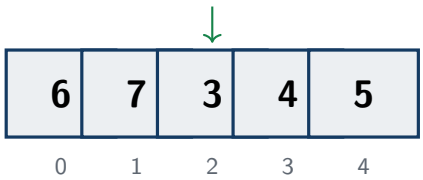
Operação	Buffer após a operação	Observação
Produz 1	`[ 1][ -][ -][ -][ -]`	escreve na posição 0
Consome 1	`[ -][ -][ -][ -][ -]`	retira da posição 0
Produz 2	`[ -][ 2][ -][ -][ -]`	
Produz 3	`[ -][ 2][ 3][ -][ -]`	
Consome 2	`[ -][ -][ 3][ -][ -]`	
Produz 4	`[ -][ -][ 3][ 4][ -]`	
Produz 5	`[ -][ -][ 3][ 4][ 5]`	
Produz 6	`[ 6][ -][ 3][ 4][ 5]`	

Esta sequência é ilustrativa: a ordem real varia conforme o escalonamento. Não é uma transcrição de uma execução observada.

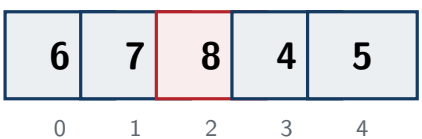
O momento da falha: um item é sobrescrito

O consumidor ainda não leu o item 3.

Produtor escreverá o item 8 na posição 2



O produtor escreve sem esperar por espaço livre:

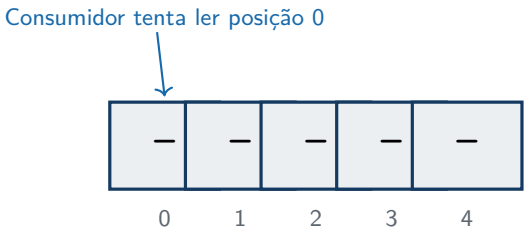


O que aconteceu com o item 3?

O item **8** ocupou a mesma posição: o item **3** foi perdido antes da leitura.

Segundo problema: e se o buffer estiver vazio?

Agora considere o consumidor mais rápido que o produtor.



Pergunta

O que o consumidor recebe se ele lê uma posição que ainda não contém um novo item?

Cenário ilustrativo: pode ocorrer na versão sem sincronização ao alterar as velocidades relativas, por exemplo, consumidor a 50 ms e produtor a 150 ms.

Buffer vazio: o consumidor lê sem esperar

Evento	Estado após o evento
CONSOME 0 ← pos. 0	[-][-][-][-][-]
CONSOME 0 ← pos. 1	[-][-][-][-][-]
PRODUZ 1 → pos. 0	[1][-][-][-][-]

```
int pos = saida;
int item = buffer[pos];   // pode ler 0: posicao vazia!
buffer[pos] = 0;
saida = (saida + 1) % TAM_BUFFER;
```

O defeito

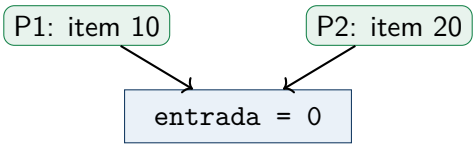
Não existe uma espera por **item disponível**. O consumidor pode avançar a posição de leitura sem ter consumido nenhum item válido.

Sequência possível, não captura de execução. Zero é marcador didático de vazio.

Terceiro problema: condição de corrida no índice

Uma pequena variação do sistema

Agora temos **dois produtores**, P1 e P2, compartilhando a variável global `entrada`. Ambos querem inserir um item.



O que muda?

Diferentemente do caso anterior, **duas threads modificam a mesma variável de posição**, sem exclusão mútua.

O trecho vulnerável: ambos usam `entrada`

```
// Executado tanto por P1 quanto por P2
int pos = entrada;                  // leitura compartilhada
buffer[pos] = item;                 // escrita no buffer
entrada = (entrada + 1) % TAM_BUFFER; // atualizacao
```

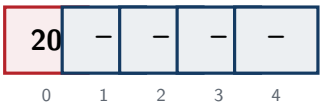
Por que isso não é uma única operação?

Uma thread pode ser interrompida entre a leitura de `entrada` e sua atualização. Nesse intervalo, outra thread pode ler o **mesmo valor**.

As duas podem escolher a mesma posição!

Intercalação: duas threads escolhem a posição 0

Passo	Produtor P1 (item 10)	Produtor P2 (item 20)
1	`pos = entrada` (lê 0)	
2		`pos = entrada` (lê 0)
3	`buffer[0] = 10`	
4		`buffer[0] = 20`
5	`entrada = 1`	
6		`entrada = 1`



Resultado desse entrelaçamento

O item **10 foi sobrescrito** por 20, e o índice avançou apenas até 1. Os dois produtores acreditavam ter reservado a mesma posição.

Três falhas, três perguntas

1. **Buffer cheio:** como impedir que o produtor sobrescreva itens pendentes?
2. **Buffer vazio:** como impedir que o consumidor leia sem haver item disponível?
3. **Condição de corrida:** como garantir que dois produtores não escolham a mesma posição?

Duas necessidades distintas

Coordenação (esperar por espaço ou itens) e **exclusão mútua** (proteger atualizações concorrentes).

Em C, acessos conflitantes não atômicos e sem sincronização caracterizam uma data race e geram comportamento indefinido.

Próximos conceitos

1. **Região crítica**: que trechos acessam dados compartilhados?
2. **Exclusão mútua**: como impedir interferências perigosas?
3. **Coordenação**: como aguardar *buffer não vazio* ou *buffer não cheio*?
4. **Primitivas**: mutex e semáforos.

Mensagem principal

Ter duas threads em execução não garante que elas cooperem corretamente. **É necessário sincronizar.**

Região Crítica

Definição

Uma **região crítica** é um trecho de código que acessa recursos compartilhados e cuja execução concorrente precisa ser controlada para preservar a correção do programa.

Exemplos de recursos compartilhados:

- Variáveis e estruturas de dados em memória;
- Buffers e filas compartilhadas;
- Arquivos e dispositivos de entrada e saída;
- Estruturas internas do Sistema Operacional.

Problema

A execução concorrente de operações sobre um recurso compartilhado pode produzir resultados incorretos quando não existe sincronização adequada.

Região Crítica: Produtor-Consumidor

Produtor

```
int posicao = entrada;

buffer[posicao] = item;

entrada = (entrada + 1)
          % TAM_BUFFER;
```

Consumidor

```
int posicao = saida;

int item = buffer[posicao];

buffer[posicao] = 0;

saida = (saida + 1)
        % TAM_BUFFER;
```

O que existe em comum?

Ambas as threads acessam o mesmo `buffer`.
Se acessarem simultaneamente a mesma posição, poderão ocorrer interferências entre suas operações.

Exclusão Mútua

Definição

A **exclusão mútua** é uma propriedade de sincronização que garante que, em um determinado instante, **no máximo uma thread ou processo** execute uma região crítica protegida pelo mesmo mecanismo.

Objetivo:

- Evitar acessos conflitantes simultâneos;
- Preservar a consistência dos dados compartilhados;
- Impedir interferências entre operações críticas.

Ideia fundamental

Se uma thread está executando a região crítica, outra thread que deseja executar a mesma região protegida deve aguardar.

Estrutura de uma Solução com Exclusão Mútua

Estrutura geral

```
while (1) {

    /* Regiao de entrada */
    /* REGIAO CRITICA */
    /* Regiao de saida */
    /* Regiao restante */

}
```

- **Região de entrada:** solicita permissão para acessar a região crítica.
- **Região crítica:** acessa ou modifica os recursos compartilhados.
- **Região de saída:** libera a região crítica.
- **Região restante:** executa operações que não precisam daquela exclusão mútua.

Exclusão Mútua é Suficiente?

Considere novamente o problema do produtor-consumidor.

Mesmo que produtor e consumidor nunca acessem simultaneamente o buffer:

- O produtor ainda pode tentar inserir um item quando o buffer estiver **cheio**.
- O consumidor ainda pode tentar retirar um item quando o buffer estiver **vazio**.

Conclusão

Exclusão mútua não é suficiente!
Além de controlar o acesso concorrente, precisamos **coordenar a execução** das threads de acordo com o estado do buffer.

Taxonomia das Soluções de Exclusão Mútua

Critério 1: Como a thread espera?

- **Espera ocupada (busy waiting):** repete testes usando CPU.
- **Espera com bloqueio:** suspende a execução até poder prosseguir.

Critério 2: Qual é o mecanismo empregado?

- **Algoritmos clássicos por software:** alternância estrita e Peterson.
- **Primitivas atômicas:** Test-and-Set, Compare-and-Swap.
- **Abstrações de biblioteca/SO:** mutexes e outras primitivas.

Atenção

Os dois critérios são **independentes**: um mutex pode usar operações atômicas e, dependendo da implementação, bloquear a thread.

Primeira Tentativa: Variável de Controle

Considere uma variável compartilhada:

Pseudocódigo

```
int ocupado = 0;
while (ocupado == 1);
ocupado = 1;
/* Regiao critica */
ocupado = 0;
```

- `ocupado = 0`: região crítica livre.
- `ocupado = 1`: região crítica ocupada.
- A thread espera enquanto a região estiver ocupada.

Pergunta

Essa solução realmente garante exclusão mútua?

Por Que a Primeira Tentativa Falha?

Suponha inicialmente:

$$ocupado = 0$$

Passo	Thread 1	Thread 2
1	Lê ocupado $= 0$	
2		Lê ocupado $= 0$
3	ocupado $= 1$	
4		ocupado $= 1$
5	Entra na região crítica	
6		Entra na região crítica

Falha

As duas threads entram na região crítica!
A verificação e a atualização da variável ocupado não formam uma operação atômica.

Espera Ocupada (Busy Waiting)

Definição

A thread verifica repetidamente uma condição para tentar entrar na região crítica, sem se bloquear voluntariamente.

Exemplo conceitual

```
while (ocupado)
    ;  /* continua testando */
```

- Consome tempo de CPU durante a espera.
- Aparece na primeira tentativa, na alternância estrita, em Peterson e nos spinlocks.
- **Não implica, por si só, incorreção:** Peterson é correto sob as hipóteses do modelo teórico.

Requisitos de uma Solução Correta

Uma solução clássica para o problema da região crítica deve satisfazer três propriedades.

1. Exclusão Mútua

No máximo uma thread pode executar a região crítica em determinado instante.

2. Progresso

Se nenhuma thread está na região crítica, a escolha de quem entrará não pode ser adiada indefinidamente pelas threads que não desejam entrar.

3. Espera Limitada

Após uma thread solicitar entrada, deve existir um limite para o número de entradas das outras threads antes dela.

Segunda Tentativa: Alternância Estrita

Considere duas threads identificadas por `0` e `1`.

Variável compartilhada

```
int vez = 0;
```

Thread 0

```
while (vez != 0);
/* Regiao critica */
vez = 1;
```

Thread 1

```
while (vez != 1);
/* Regiao critica */
vez = 0;
```

Ideia

As threads recebem permissão para entrar na região crítica alternadamente, com **espera ocupada**.

O Problema da Alternância Estrita

Suponha que:

$$vez = 0$$

1. A Thread 0 entra na região crítica.
2. Ao sair, executa `vez = 1`.
3. A Thread 1 não deseja entrar na região crítica.
4. A Thread 0 tenta entrar novamente.
5. A Thread 0 permanece esperando, pois `vez = 1`.

Problema

A Thread 0 fica impedida de entrar, mesmo com a região crítica livre.

A solução garante exclusão mútua, mas não satisfaz o requisito de progresso.

Algoritmo de Peterson

O algoritmo de Peterson utiliza duas informações:

- Se uma thread deseja entrar na região crítica;
- Qual thread recebe prioridade em caso de disputa.

Variáveis compartilhadas

```
bool interesse[2] = {false, false};

int vez = 0;
```

- `interesse[i]` indica que a thread `i` deseja entrar.
- `vez` é utilizada para resolver a disputa quando ambas desejam entrar.

Algoritmo de Peterson: Pseudocódigo

Para a thread `i`, com $j = 1 - i$:

Algoritmo teórico

```
interesse[i] = true;
vez = j;
while (interesse[j] && vez == j)
    ;
/* Regiao critica */
interesse[i] = false;
```

- A thread anuncia seu interesse e cede prioridade à outra.
- O laço `while` é uma forma de **espera ocupada**.
- Ao sair, a thread retira seu interesse.

Por Que Peterson Funciona?

Considere duas threads tentando entrar simultaneamente.

1. Ambas indicam interesse em entrar.
2. Ambas atualizam a variável vez.
3. A última atualização determina qual thread deverá ceder prioridade.
4. Uma thread entra na região crítica.
5. A outra aguarda até que a primeira retire seu interesse.

Resultado

No modelo clássico de memória sequencialmente consistente, Peterson satisfaz exclusão mútua, progresso e espera limitada para duas threads.

Das Soluções Clássicas ao OpenMP

O algoritmo de Peterson demonstra como garantir exclusão mútua utilizando variáveis compartilhadas.

Entretanto, sua utilização prática apresenta limitações:

- A versão estudada funciona com duas threads.
- Utiliza espera ocupada.
- Depende de garantias de atomicidade e ordenação.
- Exige implementar o protocolo de sincronização.

Pergunta

Precisamos implementar algoritmos como Peterson sempre que quisermos proteger uma região crítica?

Introdução ao OpenMP

Definição

OpenMP é uma API para programação paralela em memória compartilhada, utilizada com C, C++ e Fortran.

Características:

- Permite criar regiões de execução paralela.
- Utiliza threads que compartilham memória.
- Oferece diretivas para distribuir trabalho.
- Disponibiliza mecanismos de sincronização.

Diretivas

Em C, as diretivas OpenMP são introduzidas por `#pragma omp`.

OpenMP: Primeiro Programa

```
#include <stdio.h>
#include <omp.h>
int main(void)
{
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        printf("Thread %d\n", id);
    }
    return 0;
}
```

Compilação e execução

```
gcc -fopenmp exemplo.c -o exemplo
./exemplo
```

- `parallel`: cria uma região paralela.

Como as Threads Executam?

Uma possível saída

```
Thread 0
Thread 2
Thread 1
Thread 3
```

- Todas as threads executam o bloco delimitado pela região `parallel`.
- A ordem das mensagens não é determinada.
- As threads podem executar simultaneamente em diferentes núcleos.

Importante

A existência de múltiplas threads não garante uma ordem específica de execução.

Variáveis Compartilhadas e Privadas

```
int compartilhada = 0;
#pragma omp parallel num_threads(4)
{
    int privada = omp_get_thread_num();
    #pragma omp critical
    {
        compartilhada++;
    }
}
```

Variável compartilhada

`compartilhada` é acessada pelas quatro threads.

Variável privada

Cada thread possui sua própria variável local `privada`.

Condição de Corrida com OpenMP

Considere o seguinte programa:

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
printf("Contador = %ld\n", contador);
```

Resultado esperado:

$$contador = 1.000.000$$

Problema

As quatro threads modificam a mesma variável `contador` sem sincronização.
A operação `contador++` não é atômica.

OpenMP: Diretiva Critical

A diretiva `critical` permite proteger uma região crítica.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp critical
    {
        contador++;
    }
}
```

Funcionamento

Apenas uma thread por vez pode executar a região `critical` correspondente.

Resultado

O incremento passa a ser protegido por exclusão mútua.

Como Funciona a Diretiva Critical?

Considere quatro threads tentando executar a mesma região crítica.

Thread	Situação
Thread 0	Executando a região crítica
Thread 1	Aguardando
Thread 2	Aguardando
Thread 3	Aguardando

Quando a Thread 0 termina, outra thread pode entrar na região crítica.

Propriedade garantida

No máximo uma thread executa a mesma região crítica protegida em determinado instante.

Observação

A diretiva não estabelece uma ordem específica para a entrada das threads.

OpenMP: Diretiva Atomic

Quando precisamos proteger uma atualização simples, podemos utilizar a diretiva `atomic`.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp atomic
    contador++;
}
```

Funcionamento

A diretiva garante a atomicidade da atualização de `contador` em relação a acessos atômicos compatíveis.

- É aplicável a operações específicas.
- Pode ser mais eficiente que `critical`.
- Não protege arbitrariamente um bloco de código.

OpenMP: Diretiva Reduction

Outra possibilidade é evitar que todas as threads atualizem diretamente o mesmo contador.

```
long contador = 0;
#pragma omp parallel for reduction(+:contador)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
```

Funcionamento

Cada thread trabalha com uma cópia privada da variável de redução.
Ao final, os resultados são combinados utilizando a operação especificada.

Diferença fundamental

A redução evita a disputa direta pelo contador durante as iterações, em vez de proteger cada incremento com exclusão mútua.

Comparando as Soluções em OpenMP

Solução	**Funcionamento**	**Aplicação**
`critical`	Exclusão mútua entre regiões correspondentes	Proteção de blocos de código
`atomic`	Atomicidade de operações específicas	Atualizações simples de variáveis
`reduction`	Cópias privadas e combinação dos resultados	Somas, contagens e outras reduções

Conclusão

Nem toda sincronização precisa ser implementada por meio de uma região crítica explícita.
A escolha do mecanismo depende da operação e dos requisitos do programa.

Da exclusão mútua à coordenação

Retomando o produtor-consumidor

Proteger o acesso ao buffer evita interferências, mas não basta:

- o produtor precisa aguardar quando não há espaço;
- o consumidor precisa aguardar quando não há itens.

Nova questão

Como uma thread pode **esperar por uma condição** sem consultar a memória continuamente?

Semáforo: conceito

Definição

Um **semáforo** é uma abstração de sincronização com um contador interno e operações atômicas de espera e sinalização.

- **wait(S)** (ou **P**/**down**): obtém uma permissão; caso não esteja disponível, aguarda.
- **signal(S)** (ou **V**/**up**): devolve uma permissão e pode despertar uma thread.

Observação

No modelo com bloqueio, aguardar não exige executar continuamente o teste; detalhes dependem da implementação.

Semáforo binário e semáforo contador

Binário

- Valores lógicos: 0 ou 1.
- Pode controlar acesso exclusivo.
- Não se confunde integralmente com um mutex: não possui necessariamente a noção de *proprietário*.

Contador

- Representa várias permissões.
- Ex.: cinco posições livres no buffer.
- Uma espera consome uma permissão; um sinal a repõe.

Exclusão mútua e coordenação usam a mesma abstração, com propósitos diferentes.

Operações de semáforo: pseudocódigo

Semáforo inicializado com $S = k$

`wait(S): obter uma permissão; se indisponível, aguardar`

`signal(S): devolver uma permissão; liberar eventual espera`

- **Atomicidade:** duas threads não podem consumir a mesma permissão.
- **Contagem:** o valor inicial representa recursos/permissões disponíveis.
- **Ordem:** o semáforo não impõe, em geral, FIFO ou ausência de inanição.

Importante

A implementação interna de `wait`/`signal` não é um simples `if` seguido de incremento/decremento comum.

Semáforos POSIX com threads OpenMP

Duas APIs com papéis diferentes

OpenMP cria e organiza as threads; **POSIX** fornece os semáforos `sem_t` para controlar o acesso aos recursos.

- `sem_init`: inicializa o semáforo.
- `sem_wait`: obtém permissão ou aguarda.
- `sem_post`: libera uma permissão.
- `sem_destroy`: destrói o semáforo.

Atenção

O semáforo é compartilhado pelas threads do mesmo processo. O parâmetro `pshared=0` indica esse uso entre threads.

OpenMP + semáforo POSIX: exclusão mútua

```
sem_t mutex;
long contador = 0;
sem_init(&mutex, 0, 1);
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 100000; ++i) {
    sem_wait(&mutex);
    contador++;
    sem_post(&mutex);
}
sem_destroy(&mutex);
```

Ponto central

Inicializar com 1 permite apenas uma thread por vez na região crítica. O código completo está em `exemplos/`.

Do semáforo binário ao contador

Semáforo binário: exclusão mútua

`mutex = 1`: restringe o número de threads que entram na região protegida.

Semáforos contadores: coordenação

No produtor-consumidor com buffer de tamanho N:

- `vazios = N`: número inicial de vagas.
- `cheios = 0`: número inicial de itens prontos.

Por que precisamos dos três?

`vazios` e `cheios` coordenam a produção e o consumo; `mutex` protege as operações na estrutura compartilhada.

OpenMP + POSIX: produtor-consumidor

Produtor

```
produzir(item);
sem_wait(&vazios);
sem_wait(&mutex);
inserir(item);
sem_post(&mutex);
sem_post(&cheios);
```

Consumidor

```
sem_wait(&cheios);
sem_wait(&mutex);
item = retirar();
sem_post(&mutex);
sem_post(&vazios);
consumir(item);
```

Execução

`#pragma omp parallel sections` cria as duas tarefas concorrentes. Código integral disponível no projeto.

Experimentos com OpenMP + POSIX

1. Execute a contagem com quatro threads e semáforo binário inicializado com 1.
2. Compare com uma versão sem proteção da atualização.
3. Execute o produtor-consumidor com um buffer de cinco posições.
4. Inverta as velocidades relativas do produtor e consumidor.

Questão

Por que a correção do contador precisa de exclusão mútua, enquanto o buffer limitado também precisa de coordenação?

Quatro problemas clássicos de sincronização

1. **Produtor-consumidor**: controlar espaços e itens em um buffer limitado.
2. **Leitores-escritores**: permitir leituras concorrentes sem conflito com escrita.
3. **Filósofos jantando**: evitar impasse na aquisição de múltiplos recursos.
4. **Barbeiro dorminhoco**: organizar clientes, cadeiras e atendimento.

Pergunta para todos os casos

Quais são os recursos, quais condições precisam valer e qual é o risco de sincronização incorreta?

Produtor-consumidor: invariantes do buffer

Buffer de capacidade N

- $0 \leq itens \leq N$.
- Produzir requer uma **posição livre**.
- Consumir requer um **item disponível**.
- Alterações nos índices e nas posições precisam preservar a consistência.

3 ocupadas; 2 livres



Produtor-consumidor: solução com semáforos

Inicialização: `vazios = N`, `cheios = 0`, `mutex = 1`.

Produtor

```
produzir(item);
wait(vazios);
wait(mutex);
inserir(item);
signal(mutex);
signal(cheios);
```

Consumidor

```
wait(cheios);
wait(mutex);
item = retirar();
signal(mutex);
signal(vazios);
consumir(item);
```

Por que esta ordem?

Aguardar por espaço ou item **antes** de adquirir o mutex evita segurá-lo durante uma espera por condição.

Leitores e escritores: o problema

Regras para um recurso compartilhado

- Vários leitores podem acessar **simultaneamente**.
- Um escritor precisa de acesso **exclusivo**.
- Leitor e escritor não podem acessar ao mesmo tempo quando há escrita.

$$L + L + L \checkmark \qquad L + E \times \qquad E + E \times$$

Questão de justiça

Se leitores chegarem continuamente, um escritor pode nunca conseguir acesso?

Leitores-escritores: semáforos (preferência aos leitores)

Inicialização: `mutex = 1`, `recurso = 1`, `nLeitores = 0`.

Leitor

```
wait(mutex);
nLeitores++;
if (nLeitores == 1) wait(recurso);
signal(mutex);
ler();
wait(mutex);
nLeitores-;
if (nLeitores == 0) signal(recurso);
signal(mutex);
```

Escritor

```
wait(recurso);
escrever();
signal(recurso);
```

Limitação

Pode haver **inanição de escritores**.

Filósofos jantando: o problema

Cenário

Cinco filósofos alternam entre pensar e comer. Cada um precisa adquirir **dois garfos vizinhos** para comer.

- Cada garfo só pode estar com um filósofo por vez.
- Se todos pegarem o garfo à esquerda e aguardarem o da direita, ninguém progride.

Conceito central

Deadlock (impasse): ciclo de espera por recursos que impede o progresso dos participantes.

Filósofos: solução conceitual com semáforos

Cinco semáforos binários `garfo[0..4] = 1` e um semáforo contador `sala = 4`.

Filósofo *i* (pseudocódigo)

```
pensar();
wait(sala);
wait(garfo[i]);
wait(garfo[(i+1) % 5]);
comer();
signal(garfo[(i+1) % 5]);
signal(garfo[i]);
signal(sala);
```

Garantia e limite

Restringir a quatro filósofos disputando garfos **evita o deadlock circular clássico**; não garante, por si só, ausência de inanição.

Barbeiro dorminhoco: o problema

Barbearia

Há um barbeiro, uma cadeira de atendimento e N cadeiras na sala de espera.

- Sem clientes, o barbeiro dorme.
- Ao chegar, um cliente acorda o barbeiro ou espera, se houver cadeira.
- Se todas as cadeiras estiverem ocupadas, o cliente vai embora.

Desafio

Evitar perda de avisos de chegada, excesso de clientes na sala e conflitos na contagem de cadeiras.

Barbeiro: solução conceitual com semáforos

Inicialização: `clientes = 0`, `barbeiro = 0`, `mutex = 1`, `esperando = 0`.

Cliente

```
wait(mutex);
if (esperando == N) {
  signal(mutex); sair();
} else {
  esperando++;
  signal(clientes);
  signal(mutex);
  wait(barbeiro);
  receber_corte();
}
```

Barbeiro

```
while (true) {
  wait(clientes);
  wait(mutex);
  esperando-;
  signal(barbeiro);
  signal(mutex);
  cortar_cabelo();
}
```

Protocolo didático: o sinal libera um cliente para atendimento; conclusão e saída podem exigir um aperto de mãos adicional.

Comparando os problemas clássicos

Problema	Condição de avanço	Risco central
Produtor-consumidor	Espaço para produzir; item para consumir	Sobrescrita / leitura inválida
Leitores-escritores	Escrita exclusiva; leitura compartilhada	Inanição por política de prioridade
Filósofos	Dois recursos vizinhos disponíveis	Deadlock e inanição
Barbeiro	Cliente presente; cadeira de espera livre	Perda de coordenação / capacidade

Síntese

Sincronização combina **segurança** (não violar invariantes) e **vivacidade** (permitir progresso).

Para discutir em sala

1. Por que o mutex sozinho não resolve o produtor-consumidor?
2. Por que a preferência a leitores pode prejudicar escritores?
3. Evitar deadlock garante que ninguém espere indefinidamente?
4. No barbeiro, o que distingue **ser chamado** de **terminar o corte**?

Próxima etapa

Implementar e testar protocolos de sincronização, investigando também políticas de justiça e desempenho.

Exemplos executáveis: quatro problemas clássicos

Programa	Semáforos e propósito
Produtor–consumidor	`vazios`, `cheios`, `mutex`: recursos e exclusão
Leitores–escritores	`recurso`, `mutex_contagem`: leitores simultâneos
Filósofos	`garfos[5]`, `sala`: evitar espera circular
Barbeiro	`chegadas`, `mutex`, sinais individuais

Como os programas foram organizados?

OpenMP cria as threads; semáforos POSIX controlam quem pode prosseguir.

Leitores e escritores: como compartilhar a leitura?

Entrada de um leitor

```
sem_wait(&mutex_contagem);
leitores_ativos++;
if (leitores_ativos == 1) sem_wait(&recurso);
sem_post(&mutex_contagem);
```

Escritor

```
sem_wait(&recurso);
dado = novo_valor;
sem_post(&recurso);
```

Pergunta

Por que somente o primeiro leitor bloqueia o escritor? Uma sequência contínua de leitores pode causar inanição do escritor.

Filósofos: por que limitar a sala?

Filósofo *i*

```
sem_wait(&sala);
sem_wait(&garfos[i]);
sem_wait(&garfos[(i+1)%5]);
comer();
sem_post(&garfos[(i+1)%5]);
sem_post(&garfos[i]);
sem_post(&sala);
```

Inicialização

Cinco garfos com valor 1; semáforo `sala` com valor 4.

Questão

Por que permitir no máximo quatro filósofos impede o deadlock circular clássico, mas não garante ausência de inanição?

Barbeiro dorminhoco: eventos e capacidade

- `mutex = 1`: protege a fila e o número de cadeiras ocupadas.
- `chegadas = 0`: conta clientes aguardando chamada.
- `chamado[id] = 0`: acorda o cliente correto.
- `concluido[id] = 0`: sinaliza que o corte terminou.

Fluxo

Cliente entra na fila $\rightarrow$ sinaliza chegada $\rightarrow$ espera chamada $\rightarrow$ espera conclusão.

Questão

Por que um único sinal de "barbeiro disponível" pode não identificar qual cliente será atendido?

Onde aparecem os problemas clássicos?

Os mesmos padrões aparecem em sistemas reais

- **Produtor–consumidor:** filas de impressão, streaming e processamento de requisições.
- **Leitores–escritores:** caches compartilhados, bancos de dados e sistemas de arquivos.
- **Filósofos jantando:** transações e threads que precisam adquirir vários recursos.
- **Barbeiro dorminhoco:** servidores com fila limitada e serviços que aguardam solicitações.

Ideia

Os personagens mudam, mas os desafios continuam: concorrência, coordenação e acesso seguro a recursos.

Produtor–consumidor em sistemas reais

Exemplo 1: fila de impressão

Aplicações enviam documentos (produtores); um serviço de impressão retira e processa trabalhos (consumidor).

Exemplo 2: streaming de vídeo

Uma thread recebe dados da rede; outra os decodifica. Um buffer absorve diferenças momentâneas de velocidade.

Exemplo 3: servidor com workers

Uma thread recebe requisições e outras retiram tarefas de uma fila limitada.

Pergunta

Se a fila estiver cheia, o que acontece com a thread produtora na nossa solução?

Leitores–escritores em sistemas reais

Exemplo 1: cache compartilhado

Muitas threads consultam dados em paralelo; atualizações exigem coordenação com as leituras.

Exemplo 2: banco de dados

Consultas e alterações concorrentes precisam preservar a consistência dos dados.

Exemplo 3: metadados de arquivos

Várias threads consultam informações; modificações precisam ser sincronizadas.

Atenção

Bancos de dados e sistemas de arquivos reais podem usar técnicas mais sofisticadas, como MVCC e bloqueios por granularidade.

Filósofos jantando em sistemas reais

Exemplo 1: transferência entre contas

Uma transação bloqueia a conta A e espera B; outra bloqueia B e espera A. Pode ocorrer **deadlock**.

Exemplo 2: aquisição de múltiplos mutexes

Thread 1 adquire `mutex1` antes de `mutex2`; thread 2 faz o contrário.

Estratégia prática

Estabelecer uma **ordem global de aquisição** dos recursos evita a formação de espera circular.

Pergunta

Por que todas as threads adquirirem os mutexes na mesma ordem ajuda a evitar deadlock?

Barbeiro dorminhoco em sistemas reais

Exemplo 1: servidor com fila limitada

Uma thread atende requisições; outras aguardam em fila. Se a fila estiver cheia, novas solicitações podem ser rejeitadas.

Exemplo 2: processamento de tarefas

Um worker aguarda trabalho sem consumir CPU continuamente. Uma chegada sinaliza que existe tarefa disponível.

Mapeamento da analogia

Barbeiro = worker; clientes = solicitações; cadeiras = capacidade da fila; corte = processamento.

Pergunta

Quando a fila está cheia, o sistema deve bloquear a chegada ou rejeitar o pedido?

Produtor–consumidor vs. barbeiro dorminhoco

Aspecto	**Produtor–consumidor**	**Barbeiro**
Fila/buffer limitado	Sim	Sim
Quando não há trabalho	Consumidor aguarda	Barbeiro dorme
Quando a fila enche	Produtor aguarda	Cliente pode ir embora
Principal ênfase	Disponibilidade de itens e vagas	Eventos de atendimento e capacidade

Discussão

Bloquear a solicitação e rejeitá-la são duas políticas possíveis de controle de sobrecarga.

Síntese: um mesmo servidor pode reunir os quatro

1. **Produtor–consumidor:** requisições entram em uma fila e são processadas por workers.
2. **Leitores–escritores:** requisições consultam ou atualizam um cache compartilhado.
3. **Filósofos:** algumas operações exigem vários locks e podem gerar deadlock.
4. **Barbeiro:** ao atingir a capacidade da fila, o servidor pode rejeitar novas solicitações.

Questão para a turma

Quais desses padrões poderiam ocorrer simultaneamente em um único servidor web?

Atividade de observação dos programas

1. Compile os cinco programas com `make` na pasta `exemplos`.
2. Execute cada programa pelo menos três vezes.
3. Compare a ordem das mensagens e o estado dos recursos.
4. Identifique o semáforo que protege dados e o que coordena eventos.
5. Discuta deadlock, inanição e limitações de justiça de cada política.

Discussão

O resultado correto depende da sincronização; a ordem exata de execução pode variar.

Continuando: como escolher um mecanismo de sincronização?

O que já sabemos

Condições de corrida, regiões críticas, Peterson, OpenMP, semáforos e problemas clássicos.

O que vamos investigar agora

- Como as operações **atômicas** podem implementar locks?
- Em que **mutex**, **barreira** e **semáforo** diferem?
- Como **monitores** organizam exclusão mútua e espera por condições?
- Como dois **processos independentes** sincronizam o acesso à memória?

Limite desta aula

Deadlock aparece como motivação; seus algoritmos de prevenção, evitação e detecção serão estudados em aula específica.

Uma inversão perigosa no produtor–consumidor

Ordem correta: produtor

```
sem_wait(&vazios);
sem_wait(&mutex);
buffer[entrada] = item;
entrada = (entrada+1)%CAP;
sem_post(&mutex);
sem_post(&cheios);
```

Ordem incorreta

```
sem_wait(&mutex);
sem_wait(&vazios);
buffer[entrada] = item;
entrada = (entrada+1)%CAP;
sem_post(&mutex);
sem_post(&cheios);
```

A diferença está apenas na ordem de aquisição!

Como surge o impasse com buffer cheio?

1. Buffer cheio: $vazios = 0$, $cheios = CAP$, $mutex = 1$.
2. Produtor adquire `mutex` e bloqueia em `vazios`.
3. Consumidor consegue uma permissão de `cheios`, mas bloqueia em `mutex`.
4. Consumidor não retira o item nem sinaliza `vazios`.

Espera circular de dependências

Produtor espera por espaço que o consumidor criaria; consumidor espera pelo mutex mantido pelo produtor.

Importante

Os índices `entrada` e `saida` podem ser diferentes: o impasse decorre da **ordem dos semáforos**, não dos índices.

Por que o experimento precisa ser controlado?

- Uma ordem errada **permite** deadlock, mas uma execução livre pode não encontrá-lo.
- Na demonstração, começamos com o buffer **cheio** e garantimos que o produtor alcance primeiro `sem_wait(&mutex)`.
- O programa defeituoso permanece bloqueado: não devemos esperar que ele encerre sozinho.
- Para a atividade, executar com `timeout` e comparar com a versão correta.

Pergunta

Por que o consumidor não consegue liberar uma vaga se o produtor já está esperando por ela?

DEMO 01 | Produtor-consumidor: deadlock

Exemplo da Aula 3

`01_deadlock_produtor_consumidor`

No terminal, dentro da pasta do exemplo

```
make
make correto
make errado
```

O que observar

Compare o avanço na versão correta com o bloqueio da versão incorreta. O buffer começa cheio.

Atenção

`make errado` já utiliza `timeout`; não execute `./buffer errado` sem limite de tempo.

Das soluções por software ao suporte de hardware

Relembrando Peterson

Funciona como modelo conceitual para duas threads sob hipóteses apropriadas de memória; implementá-lo com variáveis comuns em C moderno não é seguro.

Primitivas atômicas

O processador fornece operações cujo efeito sobre uma posição de memória é indivisível perante operações concorrentes compatíveis.

- **Test-and-Set:** lê o valor antigo e marca o lock, atomicamente.
- **Compare-and-Swap (CAS):** substitui somente se o valor observado for o esperado.

Aplicação

Construção de locks e estruturas concorrentes; atomicidade não implica, por si só, justiça.

Spinlock: uma thread espera tentando novamente

Pseudocódigo conceitual

```
while (test_and_set(&ocupado)) {
    // espera ocupada
}
// região crítica
liberar_lock();
```

- Se o lock estava livre, a operação atômica o marca como ocupado.
- Caso contrário, a thread repete a tentativa e consome tempo de CPU.
- Pode ser útil para esperas **muito curtas**; é ruim para esperas longas.

Atenção

`test_and_set` precisa ser uma operação atômica real; um `if` seguido de atribuição não é equivalente.

Spinlock em C: operações atômicas

Exemplo ilustrativo em C11

```
atomic_flag lock = ATOMIC_FLAG_INIT;
while (atomic_flag_test_and_set(&lock)) {
    // espera ocupada
}
contador++;
atomic_flag_clear(&lock);
```

- `atomic_flag_test_and_set`: testa e marca de forma atômica.
- `atomic_flag_clear`: libera o lock.
- `contador++` fica protegido porque só uma thread possui o lock.

DEMO 02 | Spinlock com Test-and-Set

Exemplo da Aula 3

`02_spinlock`

No terminal, dentro da pasta do exemplo

```
make
./spinlock
```

O que observar

Quatro threads executam 100 mil incrementos cada. Resultado esperado: **400000**. Identifique `atomic_flag`, `test_and_set`, `clear` e a espera ocupada.

Atenção

O `while` tenta adquirir um lock; o `contador++` permanece dentro da região crítica.

CAS: outra primitiva fundamental

Compare-and-Swap (conceitualmente)

`CAS(endereco, esperado, novo)` compara o valor atual com *esperado*. Se forem iguais, grava *novo*; tudo como operação atômica.

Exemplo

Se $x = 0$, `CAS(x,0,1)` pode trocar x por 1. Se outra thread já escreveu 1, a tentativa não efetua essa troca.

- Fundamental para implementar algoritmos de sincronização.
- Uma operação atômica não resolve automaticamente problemas como inanição ou deadlock.

Mutex: o que muda em relação ao semáforo?

	Mutex POSIX	**Semáforo POSIX**
Finalidade típica	Exclusão mútua	Permissões ou eventos
Propriedade	Thread que adquire deve liberar	`sem_post` pode vir de outra thread
Operações	`pthread_mutex_lock/unlock`	`sem_wait/post`
Valor lógico	Estado bloqueado/livre	Contador de permissões

Cuidado

Semáforo POSIX inicializado com 1 não fica necessariamente restrito a 0 e 1 se forem executados `sem_post` extras.

Mutex POSIX com threads OpenMP

Exemplo ilustrativo em C/OpenMP

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
int contador = 0;
#pragma omp parallel num_threads(4)
{
    for (int i = 0; i < 10000; ++i) {
        pthread_mutex_lock(&m);
        contador++;
        pthread_mutex_unlock(&m);
    }
}
printf("contador = %d\n", contador);
```

- As threads são criadas pelo OpenMP; o mutex é fornecido pela biblioteca POSIX.
- Resultado esperado: `contador = 40000`.

DEMO 03 | Mutex POSIX + OpenMP

Exemplo da Aula 3

`03_mutex_openmp`

No terminal, dentro da pasta do exemplo

```
make
./mutex
```

O que observar

Identifique onde OpenMP cria as threads e onde `pthread_mutex_lock/unlock` protege o contador. Compare com `#pragma omp critical`.

Mutex, critical e semáforo: comparando

- `critical` **do OpenMP**: uma região nomeada ou sem nome protegida pelo runtime OpenMP.
- **Mutex POSIX**: objeto explícito com operações `lock`/`unlock` e propriedade.
- **Semáforo POSIX**: contador que também serve para sinalizar eventos entre threads.

Pergunta

Se uma thread precisa avisar *outra* que um dado ficou disponível, qual desses mecanismos corresponde mais diretamente a uma notificação?

Barreira: outro tipo de sincronização

Exclusão mútua

Impede acesso simultâneo indevido a um recurso.

Barreira

Impede que uma thread prossiga para a **próxima fase** antes que todas as threads da equipe atinjam o mesmo ponto.

Aplicações

Iterações de métodos numéricos, etapas de processamento paralelo, troca de dados entre fases.

Barreira em OpenMP

Exemplo ilustrativo em OpenMP

```
#pragma omp parallel num_threads(4)
{
    int id = omp_get_thread_num();
    printf("T%d concluiu fase 1\n", id);
    #pragma omp barrier
    printf("T%d iniciou fase 2\n", id);
}
```

- A ordem *entre* as mensagens da fase 1 varia.
- Todas as mensagens da fase 1 aparecem antes de qualquer mensagem da fase 2 (na saída sincronizada deste exemplo).

DEMO 04 | Barreira entre duas fases

Exemplo da Aula 3

`04_barreiras_openmp`

No terminal, dentro da pasta do exemplo

```
make
./barreira
./barreira
```

O que observar

As threads podem concluir a fase 1 em ordens diferentes, mas **nenhuma inicia a fase 2** antes que todas alcancem a barreira.

Barreiras: precauções

- Todas as threads da equipe devem encontrar corretamente a barreira.
- Uma barreira em caminho executado apenas por parte das threads pode causar bloqueio ou comportamento inválido.
- OpenMP também possui **barreiras implícitas** ao fim de algumas construções, salvo quando se usa `nowait` onde permitido.
- Barreira **não substitui** a proteção de uma região crítica.

Questão

Se todas as threads chegam à fase 2 juntas, elas podem atualizar a mesma variável sem proteção?

Por que introduzir monitores?

Dificuldade com semáforos

A correção depende do protocolo seguido pelo programador: ordem de `wait`, `signal` e acesso aos dados.

Ideia de monitor

Reunir **estado compartilhado** e **operações sincronizadas** em uma mesma abstração, com exclusão mútua na execução dessas operações.

Importante

Monitores estruturam a solução; não tornam automaticamente correto qualquer algoritmo de espera e sinalização.

Estrutura conceitual de um monitor

Monitor Buffer

- Estado interno: buffer, índices e quantidade de itens.
- Operações públicas: `produzir()` e `consumir()`.
- Exclusão mútua durante a execução dessas operações.
- Variáveis de condição: `naoCheio` e `naoVazio`.

Intuição

A thread solicita uma operação ao monitor; o estado compartilhado não deve ser alterado por caminhos externos que ignorem o protocolo.

Java: exclusão mútua com synchronized

Exemplo ilustrativo em Java

```
class Contador {
    private int valor = 0;
    public synchronized void incrementar() {
        valor++;
    }
    public synchronized int consultar() {
        return valor;
    }
}
```

- Métodos `synchronized` do **mesmo objeto** usam o monitor intrínseco desse objeto.
- Apenas uma thread pode possuir esse monitor de cada vez.

Variáveis de condição: esperar por mudanças

Por que não basta sincronizar um método?

No buffer limitado, produzir exige *espaço livre*; consumir exige *item disponível*. Se a condição não vale, é preciso **aguardar sem impedir o trabalho da outra thread**.

- Em Java, `wait()` libera o monitor do objeto e suspende a thread.
- `notify()` desperta uma thread aguardando naquele objeto.
- `notifyAll()` desperta todas; cada uma precisa readquirir o monitor.

Diferença importante

Ao contrário de um semáforo, uma notificação de condição não acumula permissões para o futuro.

Hoare e Mesa: quem executa após um sinal?

Semântica de Hoare (modelo clássico)

Ao sinalizar, o controle do monitor é transferido imediatamente para a thread despertada, sob a disciplina do monitor.

Semântica de Mesa (modelo usual)

A thread despertada fica apta a executar, mas precisa **readquirir o monitor**; outra thread pode alterar o estado antes disso.

Consequência prática

Em Java, sempre testar a condição em `while`, não presumir que o estado continua válido após `wait()`.

Produtor–consumidor com monitor em Java

Exemplo ilustrativo em Java

```
class Buffer {
    private int item;
    private boolean cheio = false;
    public synchronized void produzir(int x)
            throws InterruptedException {
        while (cheio) wait();
        item = x;
        cheio = true;
        notifyAll();
    }
    public synchronized int consumir()
            throws InterruptedException {
        while (!cheio) wait();
        int x = item;
        cheio = false;
        notifyAll();
        return x;
    }
}
```

O que acontece quando o consumidor chega antes?

1. Consumidor entra no método sincronizado e encontra `cheio = false`.
2. Executa `wait()`: libera o monitor e aguarda.
3. Produtor entra, deposita o item e chama `notifyAll()`.
4. Consumidor desperta, **readquire o monitor**, verifica `while` e retira o item.

E se o produtor chegar antes?

Quando o consumidor entra, encontra `cheio = true`: não precisa esperar.

DEMO 05 | Monitor produtor-consumidor em Java

Exemplo da Aula 3

`05_monitores_java`

No terminal, dentro da pasta do exemplo

```
make run
```

O que observar

Observe as mensagens de produção e consumo. No código, localize `synchronized`, `wait()` e `notifyAll()`.

Atenção

A saída não mostra necessariamente cada bloqueio em `wait()`; explique a espera pelo código e pelos slides.

Por que while e não if?

- Uma thread sinalizada precisa readquirir o monitor antes de continuar.
- Outra thread pode modificar a condição nesse intervalo.
- Podem ocorrer *despertares espúrios*, mesmo sem uma notificação correspondente.

Padrão seguro

```
while (!condicao) wait();
```

Cuidado com notify()

Se produtores e consumidores esperam no mesmo conjunto de espera, `notifyAll()` simplifica a correção do exemplo introdutório.

Monitor Java e API POSIX: correspondências

Objetivo	Java	POSIX threads
Entrar em região exclusiva	`synchronized`	`pthread_mutex_lock`
Esperar condição	`wait()`	`pthread_cond_wait`
Sinalizar uma thread	`notify()`	`pthread_cond_signal`
Sinalizar todas	`notifyAll()`	`pthread_cond_broadcast`

Não confundir

Java tem um monitor intrínseco em cada objeto; mutex + variável de condição POSIX permitem construir comportamento semelhante, mas não são uma construção *monitor* da linguagem C.

Threads e processos: qual memória é compartilhada?

Threads do mesmo processo

Compartilham o espaço de endereçamento. Um objeto `sem_t` comum pode ser acessado pelas threads.

Processos independentes

Possuem espaços de endereçamento distintos. Compartilhar uma variável exige um mecanismo explícito, como memória compartilhada POSIX.

Questão

Copiar o valor de uma variável de um processo para outro cria uma variável realmente compartilhada?

Compartilhamento entre processos: POSIX

Memória compartilhada

```
int fd = shm_open("/so_demo", O_CREAT|O_RDWR, 0600);
ftruncate(fd, sizeof(Compartilhado));
Compartilhado *p = mmap(NULL, sizeof(*p),
    PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

- `shm_open`: abre/cria objeto de memória compartilhada.
- `mmap(..., MAP_SHARED, ...)`: mapeia a mesma região nos processos.
- `munmap`, `close`, `shm_unlink`: liberam recursos.

Semáforo compartilhado entre processos

Um semáforo dentro da memória compartilhada

```
typedef struct {
    sem_t mutex;
    int contador;
} Compartilhado;

// Uma única inicialização, antes do uso concorrente:
sem_init(&p->mutex, 1, 1);
```

- `pshared = 1`: permite uso entre processos, quando suportado.
- **Não basta** usar 1: o objeto `sem_t` deve estar em memória realmente compartilhada.
- Os processos devem coordenar inicialização, uso e destruição do semáforo.

Semáforos POSIX nomeados: alternativa

Em vez de armazenar sem_t no mapeamento

`sem_open("/nome", ...)` abre um semáforo identificado por nome, acessível a processos independentes.

- `sem_wait` e `sem_post` continuam sendo as operações de espera e sinalização.
- `sem_close` fecha a referência do processo.
- `sem_unlink` remove o nome quando ele não é mais necessário.

Distinção

O semáforo coordena o acesso; ele não substitui o mecanismo usado para compartilhar os **dados**.

Exemplo: dois processos incrementam o mesmo contador

1. Um processo **inicializador** cria memória compartilhada com `contador=0` e `mutex` compartilhável.
2. Dois processos independentes abrem o mesmo objeto e mapeiam a região.
3. Cada processo repete: `sem_wait`, incrementa contador, `sem_post`.
4. Depois de ambos terminarem, o contador deve refletir a soma dos incrementos.

Ponto central

O escalonador pode intercalar os dois processos; o semáforo protege as atualizações do **mesmo objeto compartilhado**.

DEMO 06 | Dois processos, um contador

Exemplo da Aula 3

`06_processos_independentes`

No terminal, dentro da pasta do exemplo

```
make
./iniciar
./incrementar & ./incrementar & wait
./consultar
./limpar
```

O que observar

Dois processos executam 100 mil incrementos cada. Resultado esperado: **200000**. O semáforo protege o contador compartilhado.

Atenção

`wait` é do Bash, não `sem_wait()`. Limpe os recursos apenas após os processos terminarem.

Justiça, inanição e escolha dos mecanismos

O que um mecanismo garante?

- **Exclusão mútua**: impede acessos simultâneos indevidos.
- **Progresso**: alguma operação pode avançar quando há condições.
- **Espera limitada**: ninguém é ultrapassado indefinidamente.

- Semáforos POSIX não garantem, em geral, uma política FIFO.
- A solução de leitores com prioridade pode causar inanição de escritores.
- A solução dos filósofos com `sala=4` evita o impasse circular, não toda inanição.

Síntese: qual mecanismo usar?

Mecanismo	Quando é útil
Atômicas / CAS	Uma atualização curta e indivisível; base para locks.
Spinlock	Espera ocupada potencialmente curta.
Mutex	Proteger região crítica com propriedade.
Semáforo	Controlar permissões e sinalizar eventos.
Barreira	Sincronizar conclusão de fases de uma equipe.
Monitor + condição	Encapsular estado e esperar por condições sobre ele.
Memória compartilhada + semáforo	Coordenar processos com endereçamentos separados.

Pergunta final

Como decidir entre bloquear, esperar ativamente, sinalizar uma condição e sincronizar uma fase?

Leituras recomendadas e atividade

- **OSTEP**: cap. 28 (locks), 30 (variáveis de condição), 31 (semáforos); 32 (problemas de concorrência).
- **Tanenbaum e Bos, 4ª ed.**: cap. 2, seção 2.3 (sincronização e monitores).
- **Silberschatz, Galvin e Gagne, 10ª ed.**: cap. 6 (ferramentas) e 7 (exemplos de sincronização).
- **Referências de API**: documentação de OpenMP, `pthread`, POSIX `shm_open` e `sem_init`.

Atividade após a aula

Comparar a versão correta e a versão bloqueante do buffer; executar spinlock, mutex, barreira, monitor Java e dois processos que compartilham um contador.

Deadlocks (impasses)

Como reconhecer, prevenir, evitar e tratar

Ligação com as aulas anteriores

Já utilizamos mutexes, semáforos e monitores para coordenar threads. Agora veremos como a **ordem de aquisição** de recursos pode impedir o progresso.

Objetivo prático

Ler códigos curtos em C, explicar o impasse e identificar uma correção.

O que é deadlock?

Definição

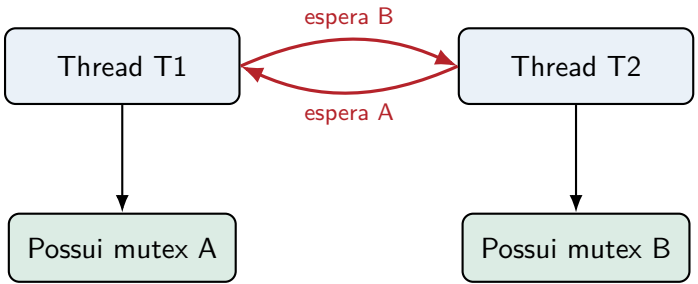
Conjunto de processos ou threads bloqueados, cada um aguardando um evento que depende de outro processo ou thread do próprio conjunto.

- Uma thread segura o recurso A e espera pelo recurso B.
- Outra segura B e espera por A.
- Nenhuma das duas consegue liberar o recurso de que a outra precisa.

Importante

Deadlock não é apenas um programa lento: **sem intervenção externa, os envolvidos não progridem**.

Um exemplo simples: dois recursos



Pergunta

O que precisaria acontecer para T1 continuar? Por que isso não acontece?

Deadlock, inanição e livelock

Situação	O que ocorre?
Deadlock	Threads ficam bloqueadas esperando umas pelas outras.
Inanição	Uma thread pode esperar indefinidamente enquanto outras continuam avançando.
Livelock	Threads continuam executando ações, mas não concluem seu trabalho.

Questão

Um programa que consome 100% de CPU necessariamente está em deadlock? **Não.**

As quatro condições necessárias (Coffman)

1. **Exclusão mútua:** pelo menos um recurso é não compartilhável.
2. **Posse e espera:** um processo mantém recursos enquanto aguarda outros.
3. **Ausência de preempção:** os recursos não podem ser retirados à força de modo seguro.
4. **Espera circular:** existe uma cadeia fechada de dependências.

Regra

Se qualquer uma dessas condições for impossível, o deadlock é impedido nesse modelo.

Condição 1: exclusão mútua

- O recurso só pode estar atribuído a um processo/thread por vez.
- Exemplo: um mutex adquirido por uma thread.
- Outro exemplo: acesso exclusivo a um dispositivo não compartilhável.

Nem todo recurso precisa ser exclusivo

Arquivos somente para leitura ou dados imutáveis podem ser compartilhados sem esse tipo de disputa.

Pergunta

Compartilhar um recurso sempre é possível? Não: depende da natureza da operação.

Condição 2: posse e espera

Situação

T1 já adquiriu o mutex A. **Sem liberar A**, tenta adquirir o mutex B.

$$\texttt{lock(A)} \longrightarrow \texttt{lock(B)}$$

- T1 possui A.
- T1 permanece aguardando B.
- A continua indisponível para outra thread.

Prevenção possível

Solicitar todos os recursos de uma vez, ou liberar os que possui antes de esperar por outros (quando viável).

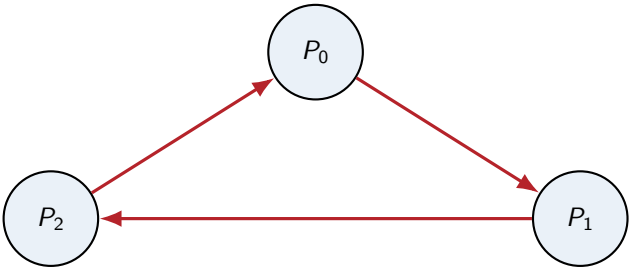
Condição 3: ausência de preempção

- Não se pode retirar arbitrariamente um recurso já adquirido.
- Um mutex não pode ser simplesmente tomado de uma thread que está no meio de uma atualização.
- Alguns recursos permitem salvamento e restauração; outros, não.

Por que isso importa?

Se fosse possível retirar um recurso e devolvê-lo depois, algumas cadeias de espera poderiam ser quebradas.

Condição 4: espera circular



Cada processo aguarda um recurso mantido pelo próximo.

Atenção

As quatro condições são **necessárias**; sua presença não garante que um deadlock já tenha ocorrido.

Revisitando o produtor–consumidor

Nosso novo exemplo em C

Buffer circular com $N = 5$ **posições**, semáforos `mutex`, `vazios` e `cheios`; produtor e consumidor executam continuamente.

- `entrada = (entrada + 1) % N`: próxima posição de escrita.
- `saida = (saida + 1) % N`: próxima posição de leitura.
- O programa começa com o buffer **cheio**, para facilitar a demonstração.

Pergunta

Se há cinco posições ocupadas, o que acontece com o produtor ao tentar inserir um sexto item?

Produtor: ordem correta dos semáforos

Trecho de 01_produtor_consumidor.c

```
while (true) {
    sem_wait(&vazios);
    sem_wait(&mutex);

    buffer[entrada] = valor++;
    entrada = (entrada + 1) % N;

    sem_post(&mutex);
    sem_post(&cheios);
}
```

Por que funciona?

O produtor espera por uma vaga **sem segurar o mutex**. O consumidor pode retirar um item e liberar uma vaga.

Produtor: a ordem incorreta provoca impasse

Troca de duas linhas

```
while (true) {
    sem_wait(&mutex);   // adquire antes da vaga
    sem_wait(&vazios);  // pode ficar esperando

    buffer[entrada] = valor++;
    entrada = (entrada + 1) % N;

    sem_post(&mutex);
    sem_post(&cheios);
}
```

Buffer inicialmente cheio

O produtor segura `mutex` e espera uma vaga. O consumidor precisa de `mutex` para liberar essa vaga.

DEMO 07 | Buffer circular: correto e errado

Pasta SO/04_deadlocks/exemplos

```
make
timeout 7s ./01_produtor_consumidor correto
timeout 5s ./01_produtor_consumidor errado
```

Observe

No modo correto, produção e consumo continuam; no errado, o produtor fica com o mutex e o consumidor não consegue esvaziar o buffer.

Como interromper?

`Ctrl+C` ou `timeout`. Os exemplos foram escritos com `while (true)`.

Dois mutexes: a ordem importa

Thread T1

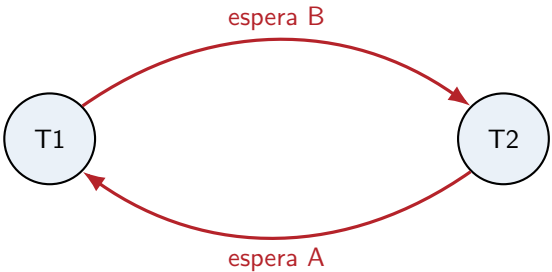
```
pthread_mutex_lock(&a);
pthread_mutex_lock(&b);
/* operacao */
pthread_mutex_unlock(&b);
pthread_mutex_unlock(&a);
```

Thread T2: ordem oposta

```
pthread_mutex_lock(&b);
pthread_mutex_lock(&a);
/* operacao */
pthread_mutex_unlock(&a);
pthread_mutex_unlock(&b);
```

T1 possui A e espera B; T2 possui B e espera A.

Grafo de dependência: dois mutexes



Todas as quatro condições estão presentes

Exclusão mútua, posse e espera, ausência de preempção e espera circular.

DEMO 08 | Dois mutexes

Pasta SO/04_deadlocks/exemplos

```
timeout 5s ./02_mutexes errado
timeout 9s ./02_mutexes correto
```

O que observar

Errado: T1 adquire A; T2 adquire B; ambas tentam o segundo mutex e param.
Correto: as duas threads usam a ordem A depois B e continuam.

Detalhe

`sleep(1)` facilita observar a intercalação, mas não é mecanismo de sincronização nem garante a ordem exata das threads.

Quatro estratégias de tratamento

Estratégia	Ideia principal
Prevenção	Impedir pelo menos uma das condições necessárias.
Evitação	Conceder recursos somente se o estado continuar seguro.
Detecção e recuperação	Permitir impasses; identificá-los e intervir.
Ignorar	Aceitar o risco quando o custo de tratar é alto.

Agora veremos

Primeiro prevenção com código; depois evitação com o algoritmo do banqueiro.

Prevenção: quebrar uma condição necessária

1. **Exclusão mútua:** tornar recursos compartilháveis quando possível.
2. **Posse e espera:** solicitar todos os recursos de uma vez ou liberar antes de solicitar outro.
3. **Ausência de preempção:** permitir retirada/restauração de recursos, quando for seguro.
4. **Espera circular:** estabelecer uma **ordem global** de aquisição.

Na prática

A ordenação global de mutexes é uma técnica simples e muito útil.

Corrigindo a ordem dos mutexes

Thread T1

```
lock(A);
lock(B);
/* operacao */
unlock(B);
unlock(A);
```

Thread T2

```
lock(A);
lock(B);
/* operacao */
unlock(B);
unlock(A);
```

Por que impede deadlock?

Ninguém pode segurar B e esperar A. A condição de **espera circular** é eliminada.

Revisitando os filósofos jantando

- Cinco filósofos e cinco garfos (um por posição na mesa).
- Para comer, cada filósofo precisa de **dois garfos**.
- Todos pegam primeiro o garfo à esquerda.
- Se todos conseguirem fazê-lo, cada um aguardará o garfo à direita.

Pergunta

Qual das condições de Coffman aparece nitidamente nessa situação?

Filósofos: duas formas de execução

Sem prevenção

```
while (true) {
  sem_wait(&garfo[i]);
  sem_wait(&garfo[(i+1)%N]);
  /* comer */
  sem_post(&garfo[(i+1)%N]);
  sem_post(&garfo[i]);
}
```

Com sala = N - 1

```
while (true) {
  sem_wait(&sala);
  sem_wait(&garfo[i]);
  sem_wait(&garfo[(i+1)%N]);
  /* comer */
  sem_post(&garfo[(i+1)%N]);
  sem_post(&garfo[i]);
  sem_post(&sala);
}
```

Limitar a disputa a $N - 1$ filósofos impede o impasse circular, mas não garante justiça.

DEMO 09 | Filósofos e prevenção

Pasta SO/04_deadlocks/exemplos

```
timeout 5s ./03_filosofos errado
timeout 12s ./03_filosofos correto
```

Observe

Errado: todos podem pegar um garfo e ficar esperando pelo outro.
Correto: a sala restringe a quantidade de filósofos que tentam pegar os garfos.

Não confundir

Ausência de deadlock **não implica** ausência de inanição.

Grafos de alocação de recursos

- **Círculo**: processo ou thread.
- **Retângulo**: tipo de recurso.
- **Processo** $\rightarrow$ **recurso**: requisição ainda não atendida.
- **Recurso** $\rightarrow$ **processo**: instância do recurso atribuída ao processo.

Regra para uma instância por tipo

Se houver ciclo nesse modelo, existe deadlock. Sem ciclo, não há deadlock.

Um grafo de alocação com ciclo



Conclusão

A e B têm uma instância cada. O ciclo representa **deadlock**.

E se houver múltiplas instâncias?

Exemplo

O recurso R possui **duas instâncias**. P_0 possui uma e espera S; P_1 possui S e espera R; P_2 possui a outra instância de R e não espera ninguém.

- Existe um ciclo envolvendo P_0, R, P_1 e S.
- Mas P_2 pode terminar e liberar uma instância de R.
- Com essa instância, P_1 pode continuar, liberar S, e P_0 avançará.

Conclusão

Com múltiplas instâncias, **um ciclo é necessário, mas não suficiente** para caracterizar deadlock no grafo de alocação.

Grafo de espera (wait-for graph)

- Simplifica a representação quando há **uma instância por tipo de recurso**.
- A aresta $P_i \to P_j$ indica que P_i espera um recurso mantido por P_j.
- Encontrar ciclo equivale a encontrar deadlock nesse modelo.

Exemplo

$P_0 \to P_1 \to P_2 \to P_0$ forma um ciclo de espera.

Cuidado

Não generalize essa conclusão diretamente para recursos com várias instâncias.

Evitação: perguntar antes de conceder

Ideia da evitação

O sistema só concede uma solicitação se, após a concessão, o estado ainda for **seguro**.

- Não basta haver recursos disponíveis neste instante.
- É necessário considerar as necessidades máximas futuras.
- O algoritmo clássico é o **algoritmo do banqueiro**.

Diferença

Prevenção restringe **como os recursos são usados**; evitação analisa **quando concedê-los**.

Estado seguro e sequência segura

Estado seguro

Existe alguma ordem na qual todos os processos podem receber os recursos que ainda podem precisar, terminar e devolver os recursos.

- Essa ordem é chamada **sequência segura**.
- Não é necessário executar os processos exatamente nessa ordem.
- Um estado **inseguro** não significa que já existe deadlock.

Princípio

O banqueiro evita concessões que levariam a estados inseguros.

Banqueiro: três processos e dez recursos

Processo	Alocado	Máximo	Ainda precisa
P_0	3	7	4
P_1	2	4	2
P_2	2	5	3

Cálculo

Disponíveis: $10 - (3 + 2 + 2) = 3$
Necessidade restante: $Need = Max - Allocation$.

Pergunta

Com 3 recursos disponíveis, qual processo consegue terminar primeiro?

Buscando a sequência segura

Etapa	Disponíveis antes	Termina	Disponíveis depois
1	3	P_1 (precisa 2)	$3 + 2 = 5$
2	5	P_0 (precisa 4)	$5 + 3 = 8$
3	8	P_2 (precisa 3)	$8 + 2 = 10$

Encontramos uma sequência segura

$P_1 \to P_0 \to P_2$. O estado inicial é **seguro**.

Por que devolvemos só o que já estava alocado?

Na simulação, os recursos adicionais seriam emprestados e devolvidos; o ganho líquido é a alocação anterior do processo.

E se concedermos mais 2 recursos a P_0?

Processo	Alocado após pedido	Máximo	Ainda precisa
P_0	5	7	2
P_1	2	4	2
P_2	2	5	3

Só resta um recurso disponível

Nenhum processo tem necessidade restante menor ou igual a 1. **Não existe sequência segura** para esse estado.

Decisão

Embora houvesse recursos para conceder o pedido, o banqueiro o **adia**.

DEMO 10 | Algoritmo do banqueiro

Pasta SO/04_deadlocks/exemplos

```
./04_banqueiro
```

- O programa testa um **estado seguro**.
- Depois simula conceder **2 recursos a** P_0.
- Mostra por que o novo estado seria **inseguro**.
- Repete os testes continuamente com `while (true)`.

Lembrar

O **teste de segurança** termina; apenas o programa de demonstração repete o teste até `Ctrl+C`.

Prevenção e evitação: não são a mesma coisa

Prevenção	**Evitação**
Impede ao menos uma condição de Coffman.	Pode manter as condições, mas controla as concessões.
Ordem global de aquisição A depois B.	Verificação de estado seguro antes de conceder.
Não precisa conhecer demandas máximas.	Precisa conhecer demandas máximas declaradas.

Limitação prática do banqueiro

Em sistemas reais, frequentemente é impossível saber antecipadamente quantos recursos cada processo poderá solicitar.

Detecção: e se o sistema permitir deadlocks?

- O sistema pode **permitir** concessões que eventualmente gerem deadlock.
- Depois, executa um mecanismo de **detecção**.
- Com recursos de instância única, podemos procurar ciclos no grafo de espera.
- Com múltiplas instâncias, é preciso considerar alocações, solicitações e recursos disponíveis.

Pergunta

Detectar um deadlock elimina o problema automaticamente? **Não.** É necessário recuperar o sistema.

Recuperação após detectar um impasse

1. **Encerrar processos:** todos os envolvidos, ou um por vez.
2. **Preemptar recursos:** quando for possível retirar e restaurar com segurança.
3. **Rollback:** restaurar um estado anterior, caso existam mecanismos de recuperação.

Critérios possíveis

Custo de reinício, recursos mantidos, trabalho já realizado e impacto sobre os usuários.

Cuidado

Liberar arbitrariamente um mutex mantido por outra thread pode corromper dados compartilhados.

Exercício rápido 1: identifique a falha

Código

```
T1:  lock(A); lock(B); trabalho; unlock(B); unlock(A);

T2:  lock(B); lock(A); trabalho; unlock(A); unlock(B);
```

1. Quais recursos são exclusivos?
2. Onde aparecem posse e espera e espera circular?
3. Que alteração simples elimina a espera circular?

Exercício rápido 2: estado seguro ou inseguro?

Situação

Há 10 recursos. P_0, P_1 e P_2 possuem, respectivamente, 3, 2 e 2 unidades. Seus máximos são 7, 4 e 5.

1. Quantos recursos estão disponíveis?
2. Qual processo pode terminar primeiro?
3. Encontre uma sequência segura.
4. Se P_0 receber mais duas unidades, ainda haverá uma sequência segura?

Distinção central

Um estado **inseguro** é necessariamente um deadlock? Justifique.

Síntese: o que precisamos saber sobre deadlocks?

- **Reconhecer** as quatro condições necessárias.
- **Desenhar** um grafo de alocação ou de espera.
- **Prevenir** por uma ordem consistente de aquisição de recursos.
- **Evitar** concessões inseguras com o algoritmo do banqueiro.
- **Distinguir** detecção de recuperação.

Quatro demonstrações em C

Buffer circular com N posições, dois mutexes, filósofos e banqueiro.

Bibliografia: deadlocks

Silberschatz, Galvin e Gagne — *Operating System Concepts*, 10ª ed. (2018)

Capítulo 8 — Deadlocks.
8.1 Modelo do sistema; 8.2 Deadlock em aplicações multithread;
8.3 Caracterização; 8.4 Métodos de tratamento; 8.5 Prevenção;
8.6 Evitação (algoritmo do banqueiro); 8.7 Detecção; 8.8 Recuperação.

Tanenbaum e Bos — *Modern Operating Systems*, 4ª ed. (2014)

Capítulo 6 — Deadlocks.
6.1 Recursos; 6.2 Introdução e modelagem; 6.3 Algoritmo do avestruz;
6.4 Detecção e recuperação; 6.5 Evitação (6.5.2 estados seguros,
6.5.3 banqueiro com um recurso, 6.5.4 com vários); 6.6 Prevenção.

Arpaci-Dusseau e Arpaci-Dusseau — *Operating Systems: Three Easy Pieces*

Capítulo 32 — Common Concurrency Problems, seção 32.3 — Deadlock Bugs.

Atividade investigativa: deadlocks

Repositório da disciplina

`SO/04_deadlocks/exemplos`

1. Execute as versões corretas e incorretas dos programas.
2. Identifique as condições de Coffman em cada deadlock.
3. Explique qual alteração no código evita cada impasse.
4. No algoritmo do banqueiro, identifique a sequência segura e explique por que a solicitação é negada.

Execução

Os exemplos concorrentes executam continuamente: encerrar com `Ctrl+C` ou `timeout`.